KEMENTERIAN PENDIDIKAN TINGGI
JABATAN PENDIDIKAN POLITEKNIK DAN KOLEJ KOMUNITI

POLITEKNIK MALAYSIA

JABATAN TEKNOLOGI MAKLUMAT DAN KOMUNIKASI

MINI PROJECT 1
SESI I : 2025/2026

DFP40443 FULL STACK WEB DEVELOPMENT

TARIKH : MINGGU KE-6
MASA : 0 JAM (F2F) + 4 JAM (NF2F)

Kertas soalan ini mengandungi **enam (6)** halaman bercetak.

DFP40443 FULL STACK WEB DEVELOPMENT

SESI 1 2026/2027

NO.	REGISTRATION NO	NAME
1	10DIT24F1503	SITI AYSHA BINTI HASSAN MOHIDEEN
2		
3		
4		

ASSESSMENT : MINI PROJECT 1

DEPARTMENT OF INFORMATION AND COMMUNICATION TECHNOLOGY | MINI PROJECT 1
SESSION: 2 2025/2026

COURSE CODE : DFP40443	**COURSE NAME :** FULL STACK WEB DEVELOPMENT

Duration : 4 Hours(NF2F)	**Total Marks _____/ 100**
CLO 1 : Integrate PHP and MySQL with Bootstrap to build database-driven web applications projects (P4, PLO3)	

INSTRUCTIONS:

1. This assessment consists of TWO (2) main practical tasks designed to evaluate your proficiency in building database-driven web applications using PHP, MySQL, and the MVC architecture.
2. You are required to submit your work in softcopy form, consisting of:
    a. **Source Code**: A compressed folder (.zip) containing all project files (Model, View, Controller folders, CSS, and JS).
    b. **Database Export**: A .sql file of your database schema.
    c. **Project Documentation**: A technical report in PDF format.
3. Read the questions carefully and refer to the attached **RUBRIC** for the marking scheme.
4. **PLAGIARISM WARNING**: Marks will be strictly deducted, or a grade of zero may be awarded, if a student is found to be copying or sharing code from/to others. All logic and implementation must be your own work.

Scenario:

As a Full Stack Web Developer, you are required to develop a Web Based PSP Student Portal. For the front-end interface, you are encouraged to utilize the Bootstrap framework via CDN to design a responsive and user-friendly layout. The system should demonstrate the implementation of:

1. MVC Architecture and modular coding
2. Session management and Secure Coding Practices
3. MySQL Database Integration and Database Relationships
4. CRUD operations

PART 1: PROFILE MANAGEMENT & ACCOUNT SECURITY

In this section, you must ensure the system can handle students' personal information securely based on an active session

1.1 Student Profile Page

- **Design:** Create a profile page that displays the student's **Name, NRIC,** and **Program**.
- **Session Management:** The information displayed must be based strictly on the currently logged-in student using the $_SESSION variable.
- **Access Control:** The system must ensure that students cannot access or view the profile information of other students

1.2 Password Exchange Module

- **Functionality:** Build a password update feature containing input fields for: **Old Password, New Password,** and **Confirm Password**.
- **Validation Logic:** The system must verify the validity of the **Old Password** against the record in the database before allowing any changes.
- **Data Security:** Students are required to use **Password Hashing** techniques to process the new password before it is stored in the database.

PART 2: GRADE MANAGEMENT SYSTEM REVAMP

You are assigned to perform a revamp so that all student data is managed entirely through a **MySQL database**.

2.1 Dynamic database (MySQL)

- implement database access using **SQL queries** to retrieve student records.

2.2 Implementation of CRUD Operations

You are required to build the interface and logic for the following operations:

Operation	Technical Requirements
CREATE	Create a form to add new student records (**Name, IC, and Marks**) directly into the database.
READ	Display a complete list of students and their marks from the database in a table format.
UPDATE	Provide functionality to update existing student information or marks.
DELETE	Provide functionality to remove student records. **Note:** Ensure there is a **confirmation** prompt before data is permanently deleted.

TECHNICAL SPECIFICATIONS

- **Architecture:** The code must strictly follow the **MVC** pattern (Model, View, Controller).
- **Modular Coding:** Use modular techniques such as include or require for reusable components like database connections.
- **Security:** Implement **Prepared Statements** for all SQL operations to prevent SQL Injection attacks.
- **Validation:** Include both client-side and server-side validation for all forms.

TABLE OF CONTENTS

1. Introduction
2. System Objectives
3. System Overview
4. System Features
5. System Architecture
6. MVC Architecture
7. MVC Project Structure
8. Entity Relationship Diagram
9. Login and Authentication
10. Session Management
11. Profile Management
12. Password Management and Security
13. CRUD Operations
14. Prepared Statements
15. Client-Side Validation
16. Server-Side Validation
17. Access Control
18. Technical Explanation of System Flow
19. Security Considerations
20. Functional Screenshots and Functional Testing
21. Conclusion

1. INTRODUCTION

The Web Based PSP Student Portal is a web application developed using PHP, MySQL, HTML, CSS, JavaScript and Bootstrap. The purpose of this project is to demonstrate the implementation of full stack web development concepts in a practical student management application.

The system provides functions for user login, student profile management, password management and student grade management. Users can securely log in to the system, view their profile information, manage student records and change their password.

The system follows the Model-View-Controller (MVC) architecture. This architecture separates the application into Models, Views and Controllers, making the application easier to understand, maintain and develop.

MySQL is used as the database management system to store user account information and student records. Bootstrap is used through CDN to provide a responsive and user-friendly interface.

The system also demonstrates secure coding practices including password hashing, password verification, session management, prepared statements, server-side validation and access control.

2. SYSTEM OBJECTIVES

The objectives of the Web Based PSP Student Portal are:

1. To develop a functional web-based student portal using PHP and MySQL.
2. To implement MVC architecture and modular coding.
3. To implement secure user authentication using PHP sessions.
4. To allow the logged-in student to view personal profile information.
5. To allow students to securely change their account password.
6. To implement CRUD operations for student records.
7. To implement MySQL database integration and relationships.
8. To use prepared statements for database operations.
9. To implement client-side validation.
10. To implement server-side validation.
11. To apply secure coding practices to protect student and account information.

3. SYSTEM OVERVIEW

The Web Based PSP Student Portal consists of several main modules.

The first module is the Login Module. Users must enter a valid email address and password before accessing the system.

After successful authentication, the user is redirected to the main student portal. The system creates a PHP session containing information about the authenticated user.

The second module is Profile Management. The logged-in user can view their Name, NRIC and Program. The profile information is retrieved from the database based on the current user's session ID.

The third module is Password Management. The user can change their password by entering the old password, new password and confirmation password. The system verifies the old password before updating the password.

The fourth module is Student Management. Users can create, read, update and delete student records. The system validates the information before performing database operations.

4. SYSTEM FEATURES

4.1 Login

The login page allows users to authenticate using their email and password.

The system checks whether the email exists in the database and then verifies the entered password against the stored password hash.

4.2 Profile Management

The profile page displays:

- Name
- NRIC
- Program

The profile information is retrieved based on the currently logged-in user's user_id.

4.3 Change Password

The Change Password module contains:

- Old Password
- New Password
- Confirm Password

The system verifies the old password before allowing the password to be changed.

The new password is hashed before being stored in the database.

4.4 Create Student

The user can create a student record containing:

- Name
- IC No
- Program
- Marks

4.5 Read Student Records

The system retrieves student records dynamically from MySQL and displays them in a table.

The table displays:

- ID
- Name
- IC No
- Program
- Marks
- Action

4.6 Update Student

The user can edit an authorised student record.

The system retrieves the selected record, displays the existing information and allows the information to be modified.

4.7 Delete Student

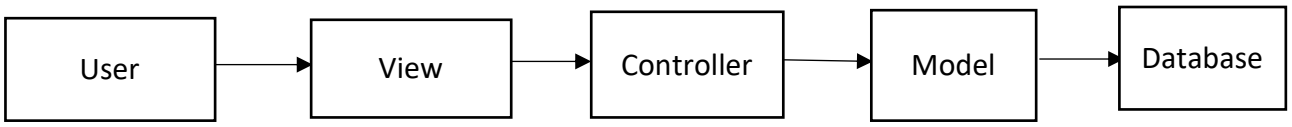
The user can delete an authorised student record.

A confirmation prompt is displayed before the delete operation is submitted.

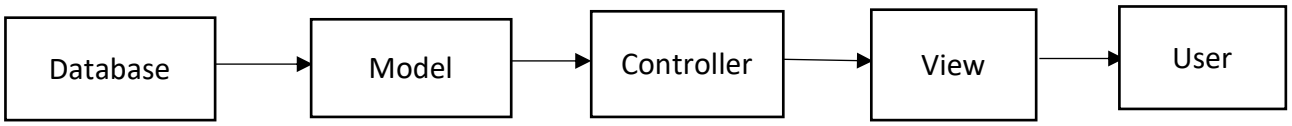
5. SYSTEM ARCHITECTURE

The application uses a three-layer MVC architecture.

The general system flow is:



The response flow is:



The View is responsible for displaying the interface.

The Controller receives user requests and controls the application flow.

The Model communicates with the MySQL database.

This separation prevents all application logic from being placed inside one PHP file.

6. MVC ARCHITECTURE

6.1 Model

The Model is responsible for database operations.

The project contains:

models/User.php

and

models/Student.php

User.php handles user-related database operations such as:

- Getting a user by ID
- Getting a user by email
- Getting student profile information
- Updating passwords

Student.php handles:

- Getting all students
- Getting a student by ID
- Getting a student based on user ID

- Creating students
- Updating students
- Deleting students

6.2 View

The View contains the user interface.

The project contains:

- login.php
- profile.php
- change_password.php
- students/create.php
- students/edit.php

The View uses Bootstrap CDN to create a responsive interface.

The View should mainly display information and forms rather than directly performing database operations.

6.3 Controller

Controllers process requests and control the application flow.

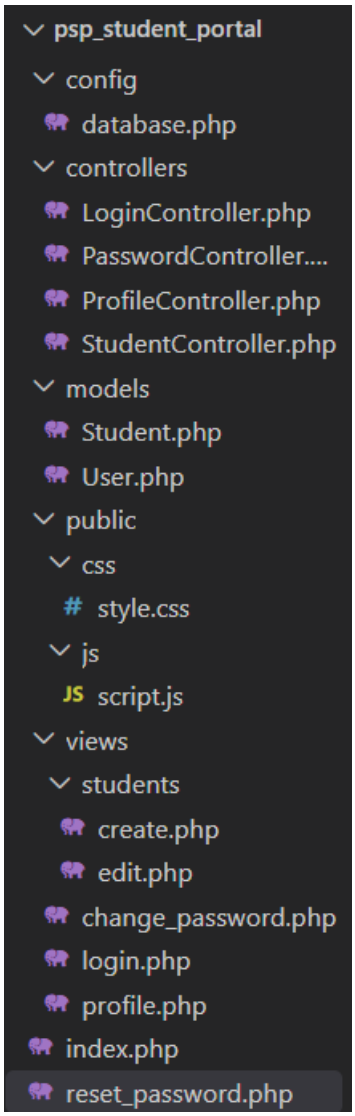
The project contains:

- LoginController.php
- ProfileController.php
- PasswordController.php
- StudentController.php

For example, LoginController.php receives the login form, validates the request and communicates with the User model.

StudentController.php handles student CREATE, UPDATE and DELETE operations.

7. MVC PROJECT STRUCTURE



The config folder contains the database connection.

The controllers folder contains application controllers.

The models folder contains database-related classes.

The views folder contains the user interface.

The public folder contains CSS and JavaScript resources.

8. ENTITY RELATIONSHIP DIAGRAM

The system contains two main entities:

USERS

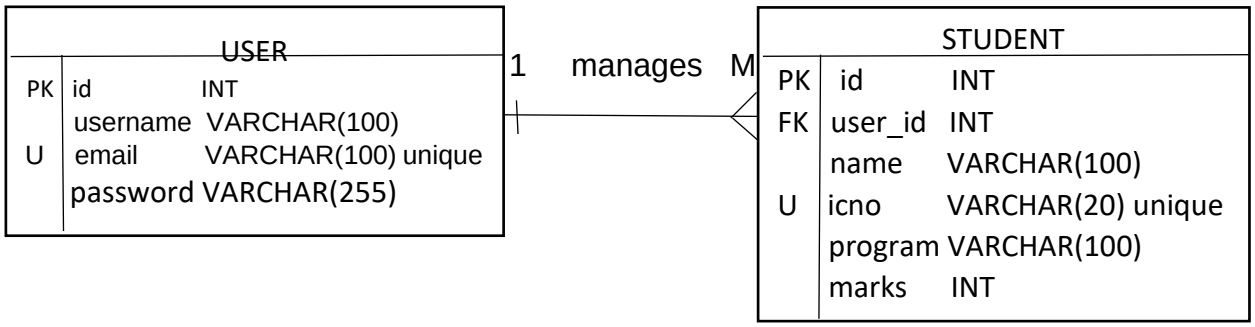
Attributes:

- id – Primary Key
- username
- email – Unique
- password

STUDENTS

Attributes:

- id – Primary Key
- user_id – Foreign Key
- name
- icno – Unique
- program
- marks

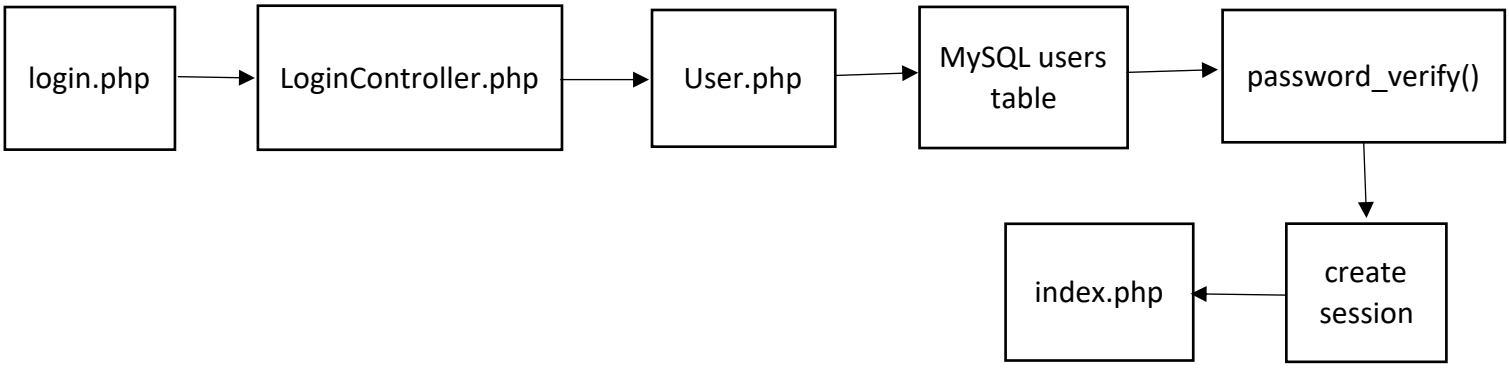


The students.user_id field references users.id.

The foreign key establishes the relationship between an account and its student records.

9. LOGIN AND AUTHENTICATION

The login process starts from:



The login form sends the email and password using the POST method.

The LoginController receives the values and performs basic validation.

The email is passed to:

$user->getByEmail($email);

The User model uses a prepared statement to retrieve the user.

The stored password hash is then checked using:

password_verify($password, $data['password']);

If authentication is successful, the system stores:

$_SESSION['user_id']
$_SESSION['username']
$_SESSION['email']

The user is then redirected to the main portal.

10. SESSION MANAGEMENT

PHP sessions are used to identify the current logged-in user.

The system checks:

```
if (!isset($_SESSION['user_id'])) {
    header("Location: ../views/login.php");
```

```
    exit;
}
```

This prevents users who are not logged in from directly accessing protected controllers.

After successful login, the application also executes:

session_regenerate_id(true);

This regenerates the session ID after authentication and helps protect against session fixation.

The session user_id is especially important because it is used to identify the current user during profile, update and delete operations.

11. PROFILE MANAGEMENT

The profile module uses the current session ID.

The ProfileController obtains:

$userId = $_SESSION['user_id'];

It then calls:

$data = $user->getStudentProfile($userId);

The User model performs a JOIN between the users and students tables.

The query uses:

WHERE users.id = ?

The ? parameter is replaced using bind_param().

The profile page displays:

- Name
- NRIC
- Program

The output is protected using:

htmlspecialchars()

This prevents database values from being interpreted as HTML code.

[INSERT PROFILE SCREENSHOT HERE]

Figure 2: Logged-in Student Profile

12. PASSWORD MANAGEMENT AND SECURITY

Password security is one of the important security features of the system.

The application does not store passwords as plain text.

When a password is created or changed, the system uses:

password_hash($password, PASSWORD_DEFAULT);

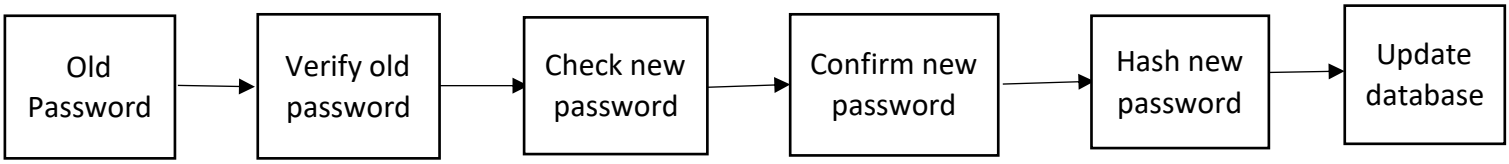
The resulting hash is stored in the database.

When logging in, the system uses:

password_verify($password, $data['password']);

This compares the entered password with the stored hash.

The change password process is:



The system also checks that the new password and confirmation password are the same.

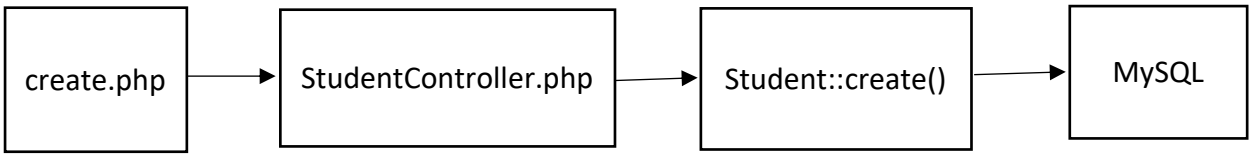
13. CRUD OPERATIONS

CRUD represents:

- Create
- Read
- Update
- Delete

13.1 CREATE

The Create operation is performed through:



The system validates the input before inserting the record.

13.2 READ

The Read operation retrieves records using:

$student->getAll();

The Student model uses a prepared statement.

The results are then displayed in the student table.

13.3 UPDATE

The Edit page retrieves the selected record.

The update request is submitted using POST.

The controller validates the information and calls:

$student->update(...)

The SQL operation also checks the user_id.

Therefore, the current user cannot update another user's record.

13.4 DELETE

The Delete operation is submitted using POST.

The form contains the student ID:

<input type="hidden" name="id" ...>

The controller then calls:

$student->delete($id, $userId);

The database query includes both:

```
WHERE id = ?
AND user_id = ?
```

A confirmation prompt is also displayed before deletion.

14. PREPARED STATEMENTS

Prepared statements are used throughout the database operations.

For example:

```
$sql = "SELECT *
        FROM users
        WHERE email = ?";

$stmt = $this->conn->prepare($sql);

$stmt->bind_param("s", $email);

$stmt->execute();
```

The ? acts as a placeholder.

The actual value is provided separately using bind_param().

This is safer than directly placing user input into an SQL string.

Prepared statements are used for:

- Login
- Profile retrieval
- Password update
- Student creation
- Student update
- Student deletion
- Student retrieval

This helps protect the application against SQL injection.

15. CLIENT-SIDE VALIDATION

Client-side validation is performed by the browser before the form is submitted.

Examples include:

required

to prevent empty fields.

The IC number uses:

pattern="[0-9]{12}"

This requires exactly 12 digits.

Marks use:

min="0"
max="100"

The password form uses:

minlength="6"

Bootstrap validation is also used to display validation messages.

Client-side validation improves user experience and allows users to identify incorrect input before submitting the form.

16. SERVER-SIDE VALIDATION

Client-side validation alone is not secure because users can bypass browser validation.

Therefore, server-side validation is also implemented in PHP.

For example, the system validates the student name using:

preg_match('/^[A-Za-z]+$/', $name)

The IC number is validated using:

preg_match('/^[0-9]{12}$/', $icno)

Marks are validated using:

if (!is_numeric($marks) || $marks < 0 || $marks > 100)

The server-side validation ensures that invalid information cannot easily be inserted into the database by bypassing the browser interface.

17. ACCESS CONTROL

Access control is implemented using the logged-in user's session.

The system obtains:

$userId = $_SESSION['user_id'];

The update query contains:

WHERE id = ?
AND user_id = ?

The delete query also contains:

WHERE id = ?
AND user_id = ?

The edit page uses:

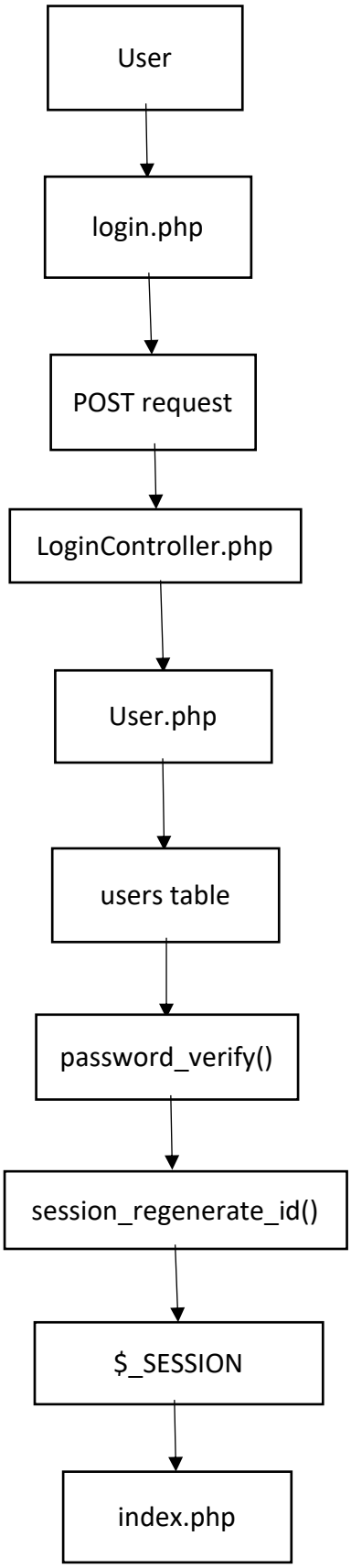
getByIdAndUser($id, $_SESSION['user_id']);

This means the system checks ownership before displaying or modifying the record.

This is important because simply hiding an Edit button is not sufficient security. A user could still manually type a URL. The database operation must also verify ownership.

18. TECHNICAL EXPLANATION OF SYSTEM FLOW

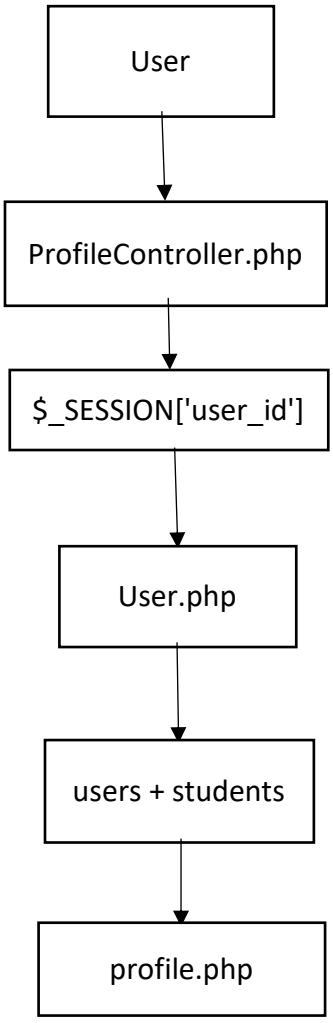
18.1 Login Flow



The controller does not directly contain the login SQL. Instead, it calls the User model.

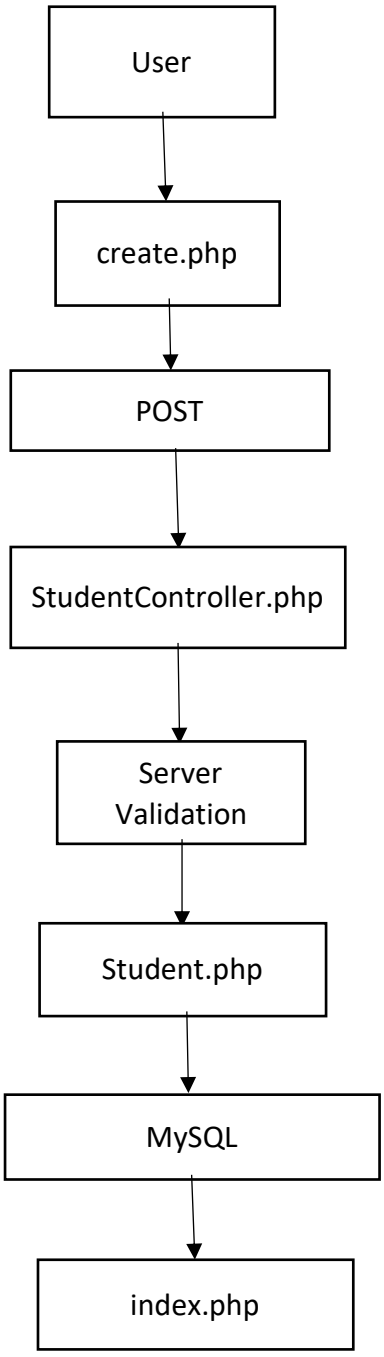
This follows MVC principles.

18.2 Profile Flow

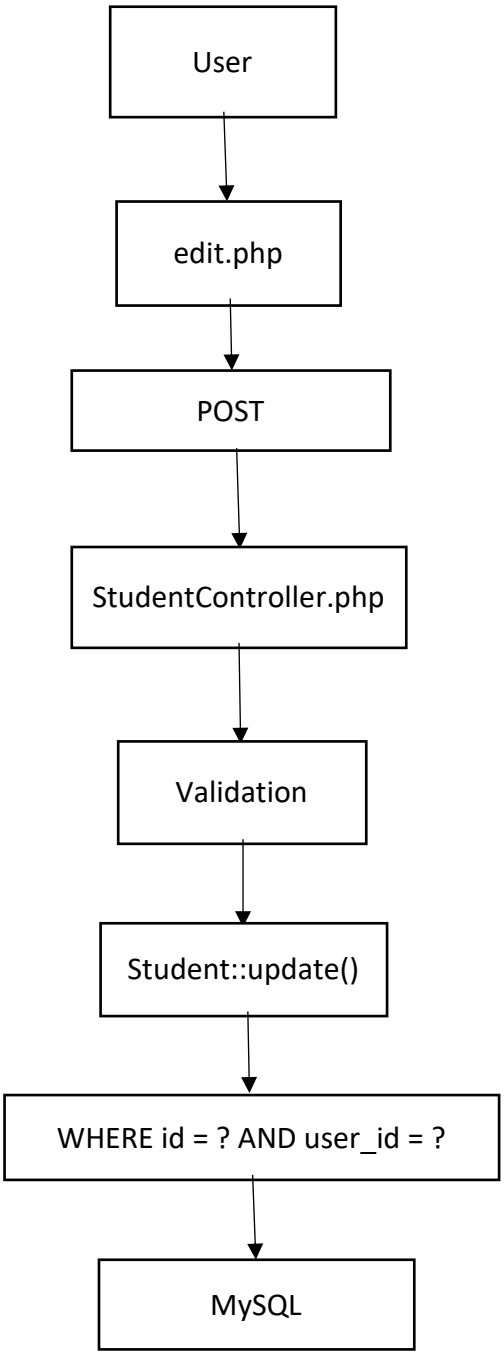


The session ID determines which profile is displayed.

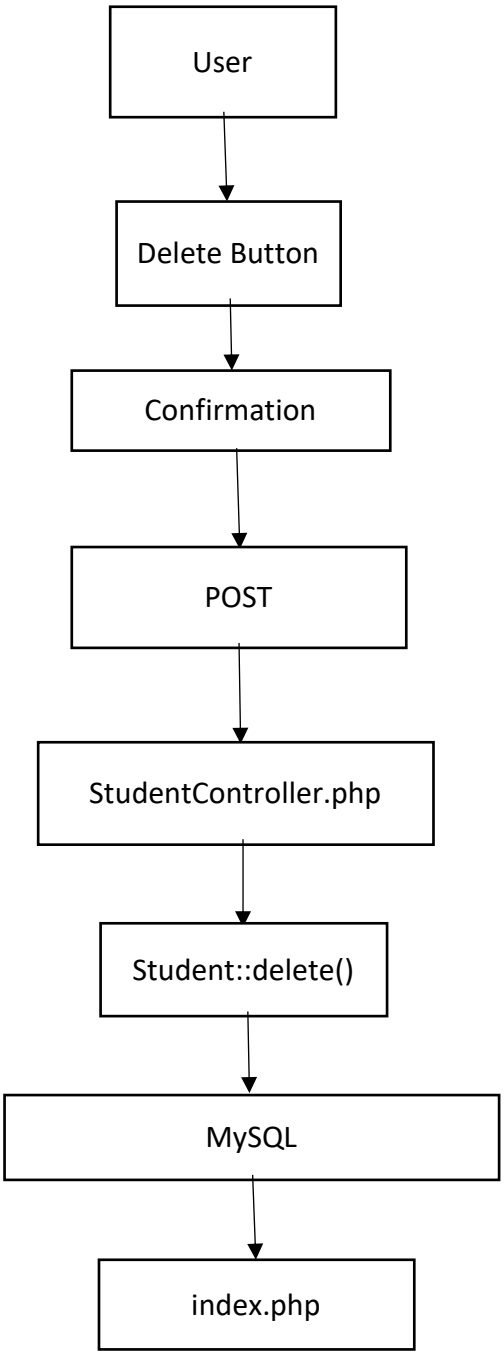
18.3 Create Flow



18.4 Update Flow



18.5 Delete Flow



19. SECURITY CONSIDERATIONS

The application implements several security practices.

19.1 Password Hashing

Passwords are stored using password_hash() rather than plain text.

19.2 Password Verification

password_verify() is used during authentication and password changes.

19.3 Prepared Statements

Prepared statements are used to reduce SQL injection risks.

19.4 Session Authentication

Protected pages check the existence of $_SESSION['user_id'].

19.5 Session Regeneration

The session ID is regenerated after successful login.

19.6 Server-Side Validation

Input is validated on the server before database operations.

19.7 Output Escaping

Database values are displayed using:

htmlspecialchars()

This helps prevent malicious HTML or script content from being interpreted by the browser.

19.8 Record Ownership

Update and delete operations verify both the student ID and current user's ID.

20. FUNCTIONAL SCREENSHOTS

CODING config/database.php

config > database.php

```php
<?php

$servername = "localhost";
$username = "root";
$password = "";
$dbname = "psp_student_portal";

$conn = new mysqli(
    $servername,
    $username,
    $password,
    $dbname
);

if ($conn->connect_error) {

    die("Database connection failed: " . $conn->connect_error);

}

$conn->set_charset("utf8mb4");

?>
```

CODING controller/LoginController.php

controllers > LoginController.php

```php
<?php

session_start();

include '../config/database.php';
include '../models/User.php';

$user = new User($conn);

if ($_SERVER['REQUEST_METHOD'] == 'POST') {

    $email = trim($_POST['email'] ?? '');
    $password = $_POST['password'] ?? '';

    if ($email == '' || $password == '') {
        die("Please fill in all fields.");
    }

    if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        die("Please enter a valid email address.");
    }

    $data = $user->getByEmail($email);

    if (!$data) {
        die("Invalid email or password.");
    }

    if (!password_verify($password, $data['password'])) {
        die("Invalid email or password.");
    }

    session_regenerate_id(true);

    $_SESSION['user_id'] = $data['id'];
```

```php
    session_regenerate_id(true);

    $_SESSION['user_id'] = $data['id'];
    $_SESSION['username'] = $data['username'];
    $_SESSION['email'] = $data['email'];

    header("Location: ../index.php");
    exit;
}

die("Invalid request.");

?>
```

CODING AND INTERFACES views/login.php

CODING

views > login.php

```
<!DOCTYPE html>

<html>

<head>

    <title>PSP Student Portal Login</title>

    <meta charset="UTF-8">

    <meta name="viewport"
        content="width=device-width, initial-scale=1">

    <link
        href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
        rel="stylesheet">

    <link
        rel="stylesheet"
        href="../public/css/style.css">

</head>

<body>

<div class="container mt-5">

    <div class="row justify-content-center">

        <div class="col-md-5">

            <div class="card">

                <div class="card-header text-center">
```

```
                <div class="card-header text-center">

                    <h3>PSP Student Portal</h3>

                </div>

                <div class="card-body">

                    <form
                        action="../controllers/LoginController.php"
                        method="POST">

                        <div class="mb-3">

                            <label class="form-label">
                                Email
                            </label>

                            <input
                                type="email"
                                name="email"
                                class="form-control"
                                required>

                        </div>

                        <div class="mb-3">

                            <label class="form-label">
                                Password
```

```
                            <label class="form-label">
                                Password
                            </label>

                            <input
                                type="password"
                                name="password"
                                class="form-control"
                                required>

                        </div>

                        <button
                            type="submit"
                            class="btn btn-primary w-100">

                            Login

                        </button>

                    </form>

                </div>

            </div>

        </div>

    </div>
```

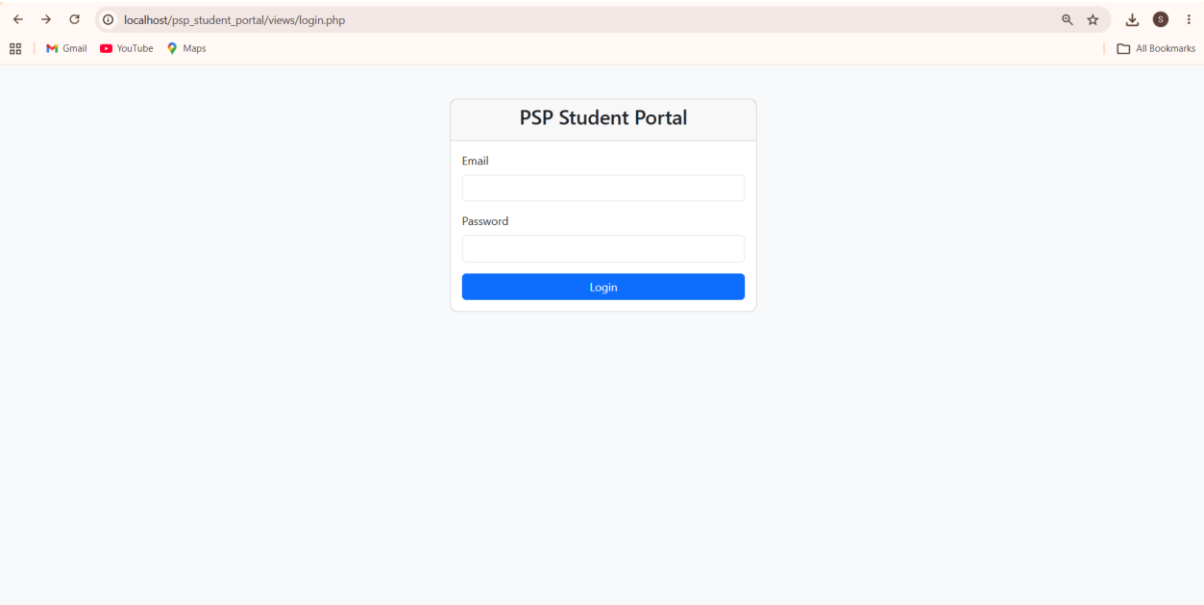
```
    </div>

</div>

</body>

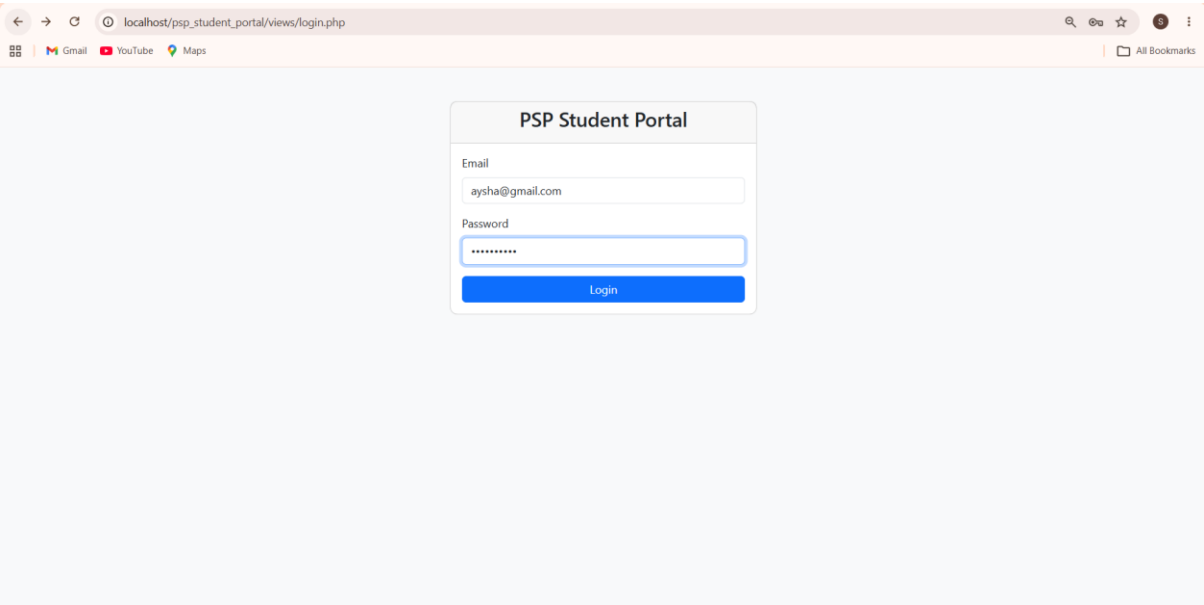
</html>
```

INTERFACE(Login Page)



Email : aysha@gmail.com

Password : Aysha@1686



CODING AND INTERFACES index.php

CODING

index.php

```php
<?php

session_start();

if (!isset($_SESSION['user_id'])) {

    header("Location: views/login.php");
    exit;
}

include 'config/database.php';
include 'models/Student.php';

$student = new Student($conn);

$result = $student->getAll();

?>

<!DOCTYPE html>

<html>

<head>

    <title>PSP Student Portal</title>

    <meta charset="UTF-8">

    <meta name="viewport"
          content="width=device-width, initial-scale=1">

    <link
        href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
        rel="stylesheet">
```

```php
    <link
        href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
        rel="stylesheet">

    <link
        rel="stylesheet"
        href="public/css/style.css">

</head>

<body>

<div class="container mt-5">

    <div class="d-flex justify-content-between align-items-center mb-4">

        <div>

            <h2>PSP Student Portal</h2>

            <p class="mb-0">

                Welcome,
                <?php echo htmlspecialchars($_SESSION['username']); ?>

            </p>

        </div>

        <div>

            <a
```

```
64          <a
65              href="controllers/ProfileController.php"
66              class="btn btn-info">
67
68              Profile
69
70          </a>
71
72          <a
73              href="views/students/create.php"
74              class="btn btn-success">
75
76              Add Student
77
78          </a>
79
80          <a
81              href="views/login.php"
82              class="btn btn-danger"
83              onclick="return confirm('Are you sure you want to logout?');">
84
85              Logout
86
87          </a>
88
89      </div>
90
91  </div>
92
93
94  <div class="card">
95
```

```
94  <div class="card">
95
96      <div class="card-header">
97
98          <h4>Student List</h4>
99
100     </div>
101
102     <div class="card-body">
103
104         <div class="table-responsive">
105
106             <table class="table table-bordered table-striped">
107
108                 <thead>
109
110                 <tr>
111
112                     <th>ID</th>
113
114                     <th>Name</th>
115
116                     <th>IC No</th>
117
118                     <th>Program</th>
119
120                     <th>Marks</th>
121
122                     <th>Action</th>
123
124                 </tr>
125
126                 </thead>
127
```

```
126                 </thead>
127
128                 <tbody>
129
130                 <?php
131
132                 if ($result->num_rows > 0) {
133
134                     while ($row = $result->fetch_assoc()) {
135
136                 ?>
137
138                 <tr>
139
140                     <td>
141                         <?php echo htmlspecialchars($row['id']); ?>
142                     </td>
143
144                     <td>
145                         <?php echo htmlspecialchars($row['name']); ?>
146                     </td>
147
148                     <td>
149                         <?php echo htmlspecialchars($row['icno']); ?>
150                     </td>
151
152                     <td>
153                         <?php echo htmlspecialchars($row['program']); ?>
154                     </td>
155
```

```
155
156                     <td>
157                         <?php echo htmlspecialchars($row['marks']); ?>
158                     </td>
159
160                     <td>
161
162                         <?php
163
164                         if ($row['user_id'] == $_SESSION['user_id']) {
165
166                         ?>
167
168                             <a
169                                 href="views/students/edit.php?id=<?php echo $row['id']; ?>"
170                                 class="btn btn-warning btn-sm">
171
172                                 Edit
173
174                             </a>
175
176                             <form
177                                 action="controllers/StudentController.php?action=delete"
178                                 method="POST"
179                                 style="display:inline;">
180
181                                 <input
182                                     type="hidden"
183                                     name="id"
184                                     value="<?php echo $row['id']; ?>">
```

```
                                value="<?php echo $row['id']; ?>">

                            <button
                                type="submit"
                                class="btn btn-danger btn-sm"
                                onclick="return confirm('Are you sure you want to delete this stude

                                Delete

                            </button>

                        </form>

                    <?php

                    } else {

                    ?>

                        <span class="text-muted">
                            View Only
                        </span>

                    <?php

                    }

                    ?>

                </td>
```

```
                </td>

            </tr>

            <?php

                }

            } else {

            ?>

            <tr>

                <td
                    colspan="6"
                    class="text-center">

                    No student records found.

                </td>

            </tr>

            <?php

            }

            ?>
```

```
                        colspan="6"
                        class="text-center">

                        No student records found.

                    </td>

                </tr>

                <?php

                }

                ?>

                </tbody>

            </table>

        </div>

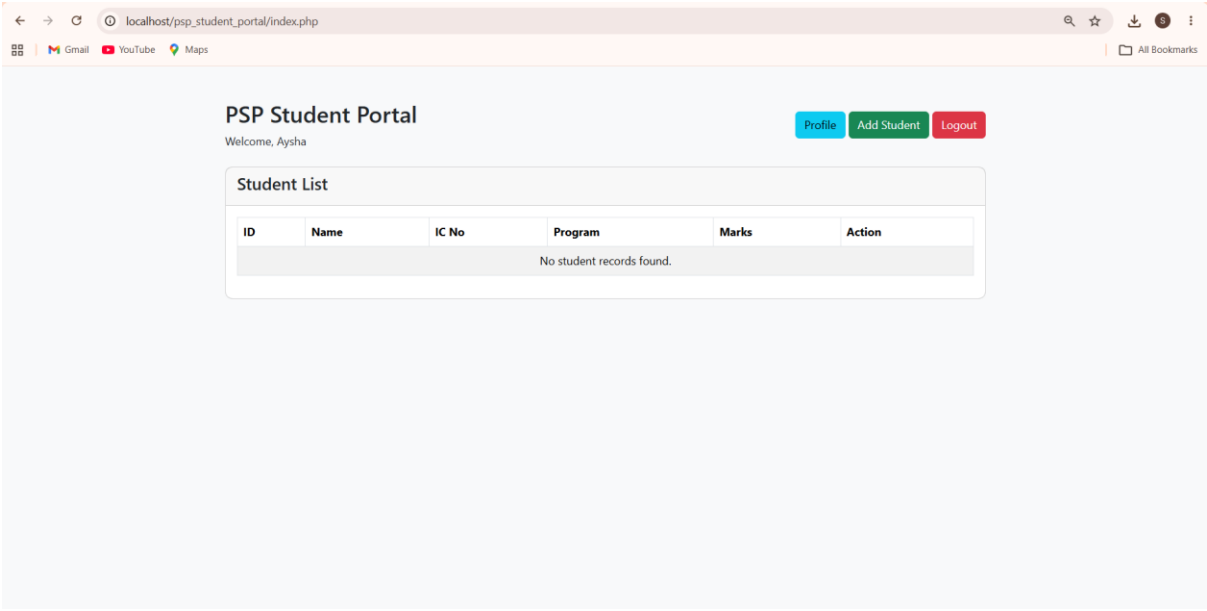
    </div>

</div>

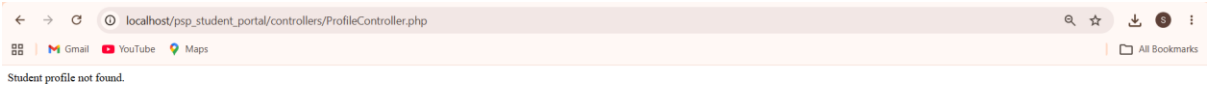
</body>

</html>
```

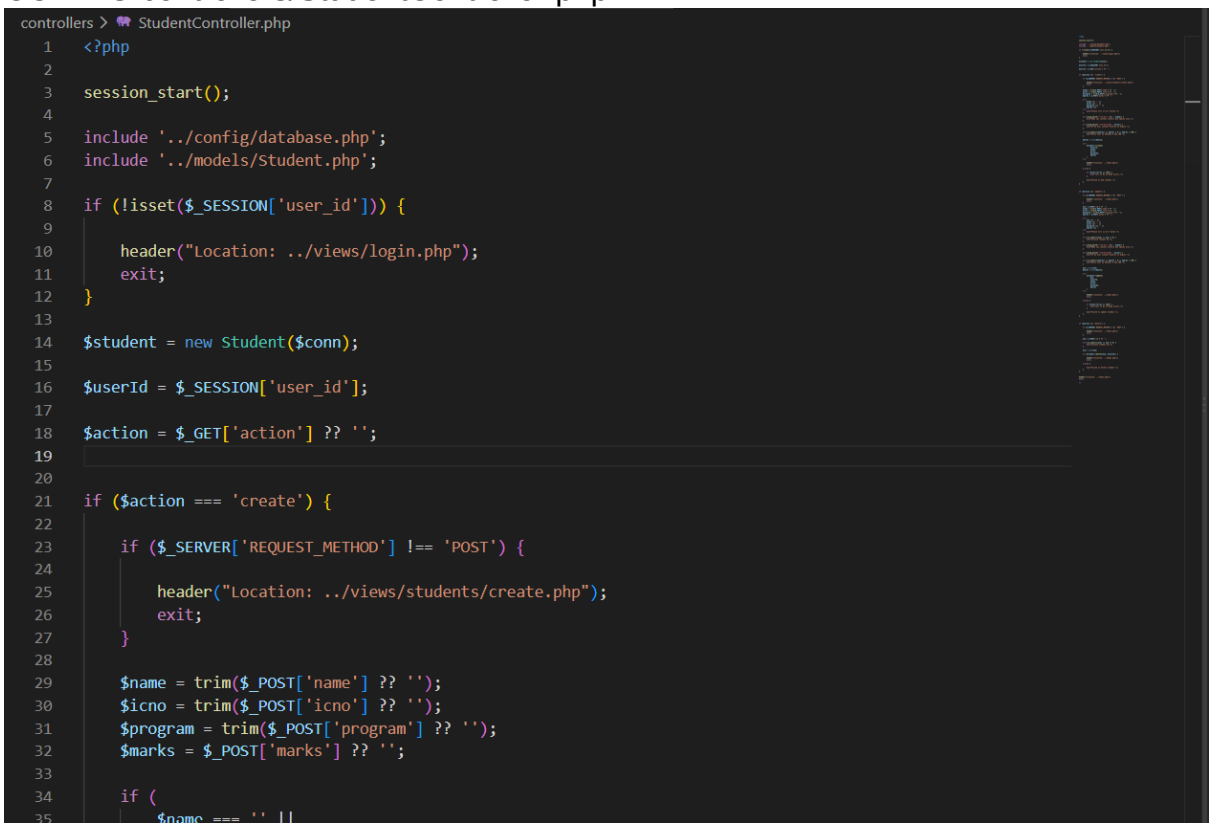
INTERFACE (Before Add Student)(Main Dashboard)



INTERFACE controller/ProfileController.php (Before add Student)



CODING controllers/StudentController.php



```php

    if (
        $name === '' ||
        $icno === '' ||
        $program === '' ||
        $marks === ''
    ) {
        die("Please fill in all fields.");
    }

    if (!preg_match('/^[A-Za-z ]+$/', $name)) {
        die("Name can contain letters and spaces only.");
    }

    if (!preg_match('/^[0-9]{12}$/', $icno)) {
        die("IC No must contain exactly 12 digits.");
    }

    if (!is_numeric($marks) || $marks < 0 || $marks > 100) {
        die("Marks must be between 0 and 100.");
    }

    $marks = (int)$marks;

    if (
        $student->create(
            $userId,
            $name,
            $icno,
            $program,
            $marks
        )
    ) {

```

```php
        )
    ) {

        header("Location: ../index.php");
        exit;

    } else {

        if ($conn->errno == 1062) {
            die("This IC No already exists.");
        }

        die("Failed to add student.");
    }
}



if ($action === 'update') {

    if ($_SERVER['REQUEST_METHOD'] !== 'POST') {

        header("Location: ../index.php");
        exit;
    }

    $id = $_POST['id'] ?? '';
    $name = trim($_POST['name'] ?? '');
    $icno = trim($_POST['icno'] ?? '');
    $program = trim($_POST['program'] ?? '');
    $marks = $_POST['marks'] ?? '';

    if (
```

```php

    if (
        $id === '' ||
        $name === '' ||
        $icno === '' ||
        $program === '' ||
        $marks === ''
    ) {
        die("Please fill in all fields.");
    }

    if (!is_numeric($id) || $id <= 0) {
        die("Invalid student ID.");
    }

    if (!preg_match('/^[A-Za-z ]+$/', $name)) {
        die("Name can contain letters and spaces only.");
    }

    if (!preg_match('/^[0-9]{12}$/', $icno)) {
        die("IC No must contain exactly 12 digits.");
    }

    if (!is_numeric($marks) || $marks < 0 || $marks > 100) {
        die("Marks must be between 0 and 100.");
    }

    $id = (int)$id;
    $marks = (int)$marks;

    if (
        $student->update(
            $id,
            $userId,
```

```php
    if (
        $student->update(
            $id,
            $userId,
            $name,
            $icno,
            $program,
            $marks
        )
    ) {

        header("Location: ../index.php");
        exit;

    } else {

        if ($conn->errno == 1062) {
            die("This IC No already exists.");
        }

        die("Failed to update student.");
    }
}



if ($action === 'delete') {

    if ($_SERVER['REQUEST_METHOD'] !== 'POST') {

        header("Location: ../index.php");
        exit;
    }

```

```php

if ($action === 'delete') {

    if ($_SERVER['REQUEST_METHOD'] !== 'POST') {

        header("Location: ../index.php");
        exit;
    }

    $id = $_POST['id'] ?? '';

    if (!is_numeric($id) || $id <= 0) {
        die("Invalid student ID.");
    }

    $id = (int)$id;

    if ($student->delete($id, $userId)) {

        header("Location: ../index.php");
        exit;

    } else {

        die("Failed to delete student.");
    }
}


header("Location: ../index.php");
exit;

?>
```

CODING AND INTERFACE views/students/create.php

CODING

```php
views > students > create.php
<!DOCTYPE html>

<html>

<head>

    <title>Add Student</title>

    <meta charset="UTF-8">

    <meta name="viewport"
        content="width=device-width, initial-scale=1">

    <link
        href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
        rel="stylesheet">

    <link
        rel="stylesheet"
        href="../../public/css/style.css">

</head>

<body>

<div class="container mt-5">

    <div class="card">

        <div class="card-header">

            <h4>Add Student</h4>

        </div>
```

```
35
36          <div class="card-body">
37
38              <form
39                  action="../../controllers/StudentController.php?action=create"
40                  method="POST"
41                  class="needs-validation"
42                  novalidate>
43
44                  <div class="mb-3">
45
46                      <label class="form-label">
47                          Name
48                      </label>
49
50                      <input
51                          type="text"
52                          name="name"
53                          class="form-control"
54                          pattern="[A-Za-z ]+"
55                          required>
56
57                      <div class="invalid-feedback">
58                          Please enter a valid name.
59                      </div>
60
61                  </div>
62
63                  <div class="mb-3">
64
65                      <label class="form-label">
66                          IC No
67                      </label>
```

```
68
69                      <input
70                          type="text"
71                          name="icno"
72                          class="form-control"
73                          pattern="[0-9]{12}"
74                          maxlength="12"
75                          required>
76
77                      <div class="invalid-feedback">
78                          IC No must contain exactly 12 digits.
79                      </div>
80
81                  </div>
82
83                  <div class="mb-3">
84
85                      <label class="form-label">
86                          Program
87                      </label>
88
89                      <input
90                          type="text"
91                          name="program"
92                          class="form-control"
93                          required>
94
95                      <div class="invalid-feedback">
96                          Please enter the program.
97                      </div>
```

```
                </div>

                <div class="mb-3">

                    <label class="form-label">
                        Marks
                    </label>

                    <input
                        type="number"
                        name="marks"
                        class="form-control"
                        min="0"
                        max="100"
                        required>

                    <div class="invalid-feedback">
                        Marks must be between 0 and 100.
                    </div>

                </div>

                <button
                    type="submit"
                    class="btn btn-success">

                    Add Student

                </button>
```

```
                <a
                    href="../../index.php"
                    class="btn btn-secondary">

                    Cancel

                </a>

            </form>

        </div>

    </div>

</div>


<script>

(function () {

    'use strict';

    var forms =
        document.querySelectorAll('.needs-validation');

    Array.prototype.slice.call(forms)
        .forEach(function (form) {

            form.addEventListener(
```

```
    var forms =
        document.querySelectorAll('.needs-validation');

    Array.prototype.slice.call(forms)
        .forEach(function (form) {

            form.addEventListener(
                'submit',
                function (event) {

                    if (!form.checkValidity()) {

                        event.preventDefault();
                        event.stopPropagation();

                    }

                    form.classList.add(
                        'was-validated'
                    );

                },
                false
            );

        });

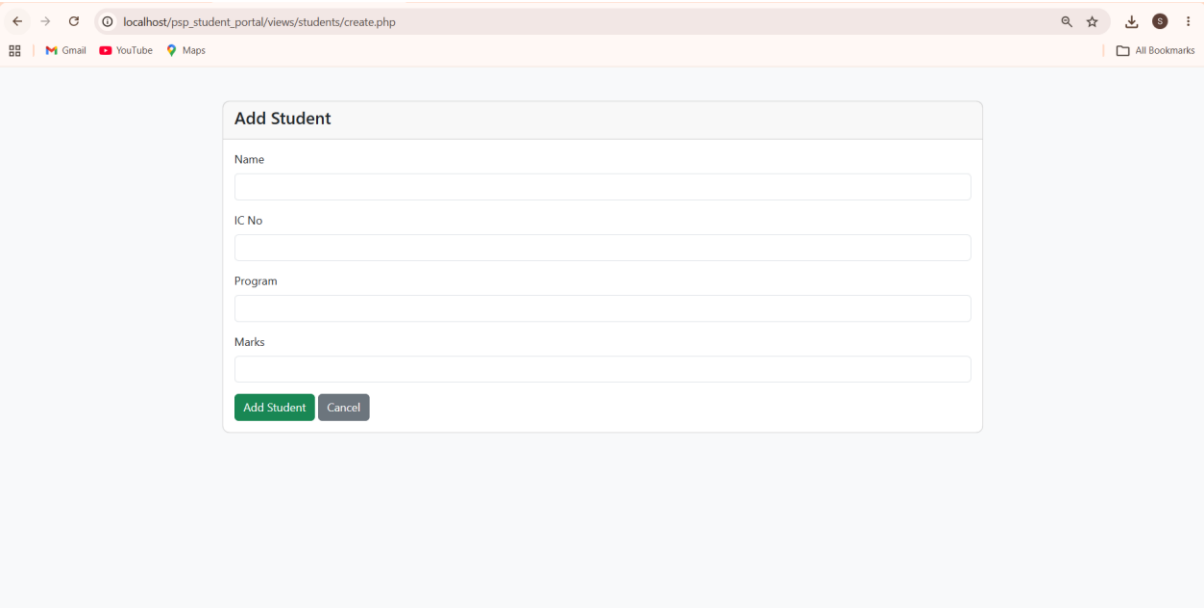
})();

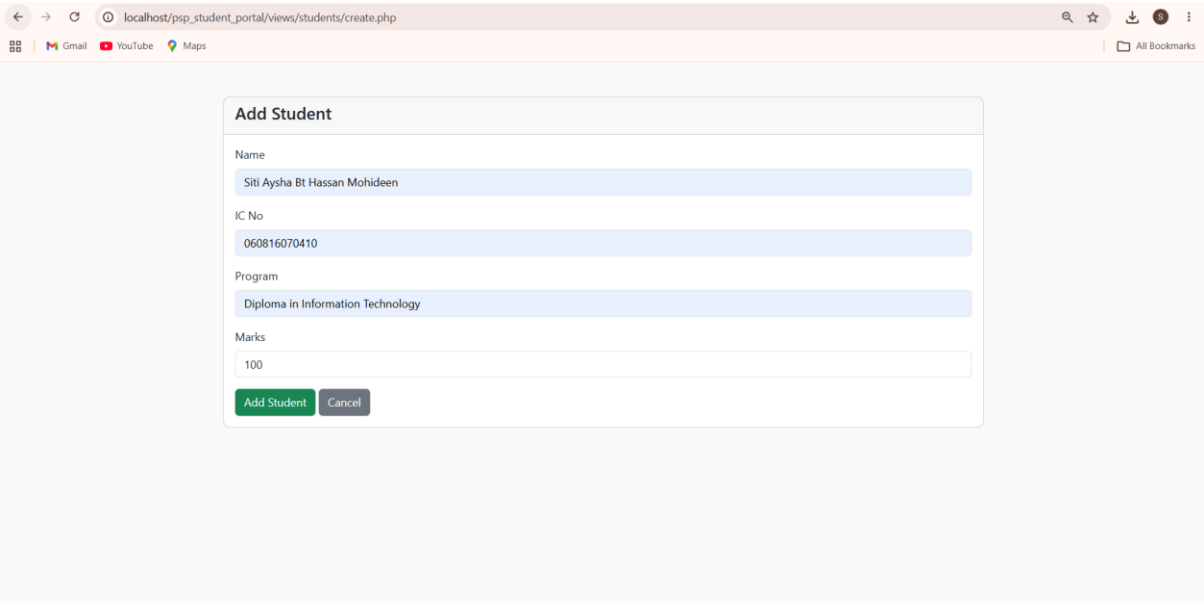
</script>

</body>

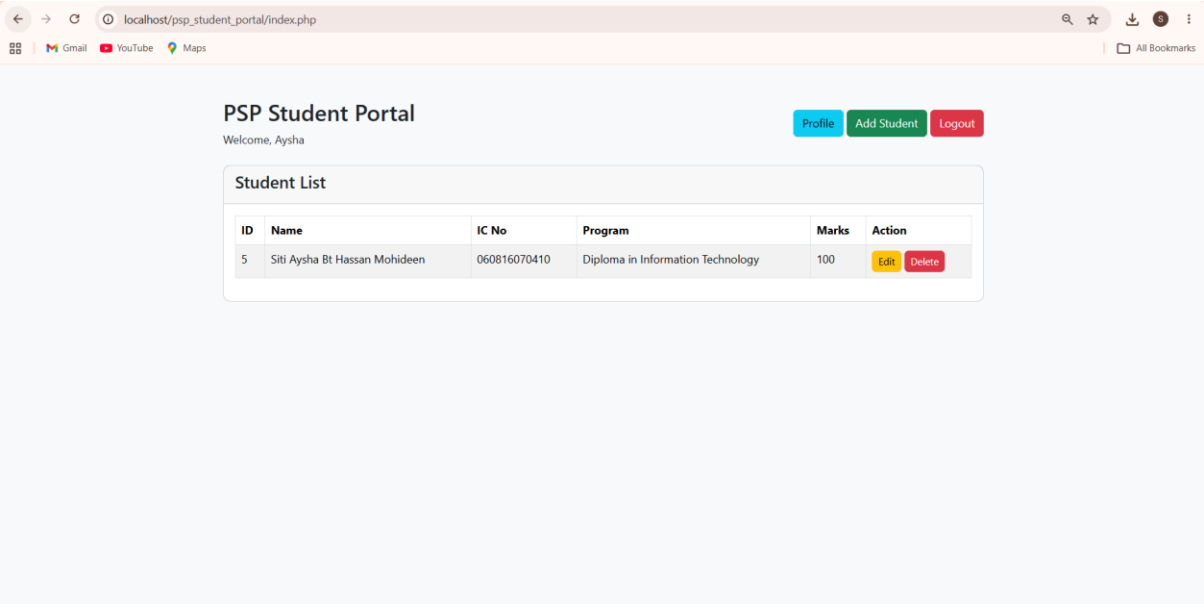
</html>
```

INTERFACE(Add Student)

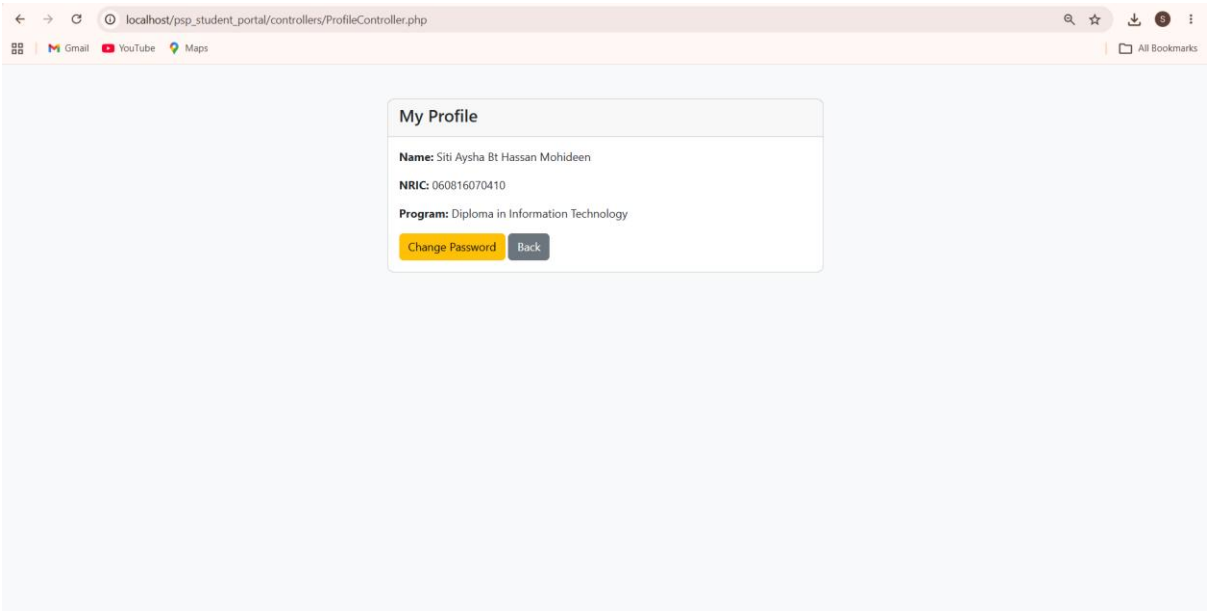




INTERFACE (after add student)(Student Record Created)



INTERFACE controllers/ProfileController.php (after add Student)



CODING AND INTERFACE views/students/edit.php?id=5

CODING

views > students > edit.php

```php
<?php

session_start();

if (!isset($_SESSION['user_id'])) {

    header("Location: ../login.php");
    exit;
}

include '../../config/database.php';
include '../../models/Student.php';

$student = new Student($conn);

$id = $_GET['id'] ?? 0;

if (!is_numeric($id) || $id <= 0) {
    die("Invalid student ID.");
}

$data = $student->getByIdAndUser(
    $id,
    $_SESSION['user_id']
);

if (!$data) {
    die("Student not found.");
}

?>

<!DOCTYPE html>

<html>
```

views > students > edit.php

```
<html>

<head>

    <title>Edit Student</title>

    <meta charset="UTF-8">

    <meta name="viewport"
        content="width=device-width, initial-scale=1">

    <link
        href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
        rel="stylesheet">

    <link
        rel="stylesheet"
        href="../../public/css/style.css">

</head>

<body>

<div class="container mt-5">

    <div class="card">

        <div class="card-header bg-primary text-white">

            <h4>Edit Student</h4>

        </div>

        <div class="card-body">
```

```
68          <div class="card-body">
69
70              <form
71                  action="../../controllers/StudentController.php?action=update"
72                  method="POST"
73                  class="needs-validation"
74                  novalidate>
75
76                  <input
77                      type="hidden"
78                      name="id"
79                      value="<?php echo htmlspecialchars($data['id']); ?>">
80
81                  <div class="mb-3">
82
83                      <label class="form-label">
84                          Name
85                      </label>
86
87                      <input
88                          type="text"
89                          name="name"
90                          class="form-control"
91                          value="<?php echo htmlspecialchars($data['name']); ?>"
92                          pattern="[A-Za-z ]+"
93                          required>
94
95                      <div class="invalid-feedback">
96                          Please enter a valid name.
97                      </div>
98
99                  </div>
100
```

```
98
99                  </div>
100
101                 <div class="mb-3">
102
103                     <label class="form-label">
104                         IC No
105                     </label>
106
107                     <input
108                         type="text"
109                         name="icno"
110                         class="form-control"
111                         value="<?php echo htmlspecialchars($data['icno']); ?>"
112                         pattern="[0-9]{12}"
113                         maxlength="12"
114                         required>
115
116                     <div class="invalid-feedback">
117                         IC No must contain exactly 12 digits.
118                     </div>
119
120                 </div>
121
122                 <div class="mb-3">
123
124                     <label class="form-label">
125                         Program
126                     </label>
127
```

```
                    <input
                        type="text"
                        name="program"
                        class="form-control"
                        value="<?php echo htmlspecialchars($data['program']); ?>"
                        required>

                    <div class="invalid-feedback">
                        Please enter the program.
                    </div>

                </div>

                <div class="mb-3">

                    <label class="form-label">
                        Marks
                    </label>

                    <input
                        type="number"
                        name="marks"
                        class="form-control"
                        value="<?php echo htmlspecialchars($data['marks']); ?>"
                        min="0"
                        max="100"
                        required>

                    <div class="invalid-feedback">
```

```
                    <div class="invalid-feedback">
                        Marks must be between 0 and 100.
                    </div>

                </div>

                <button
                    type="submit"
                    class="btn btn-primary">

                    Update Student

                </button>

                <a
                    href="../../index.php"
                    class="btn btn-secondary">

                    Cancel

                </a>

            </form>

        </div>

    </div>

</div>
```

```
</div>

<script>

(function () {

    'use strict';

    var forms =
        document.querySelectorAll('.needs-validation');

    Array.prototype.slice.call(forms)
        .forEach(function (form) {

            form.addEventListener(
                'submit',
                function (event) {

                    if (!form.checkValidity()) {

                        event.preventDefault();
                        event.stopPropagation();


                    }

                    form.classList.add(
                        'was-validated'
                    );

                },
                false
            );

        });
```

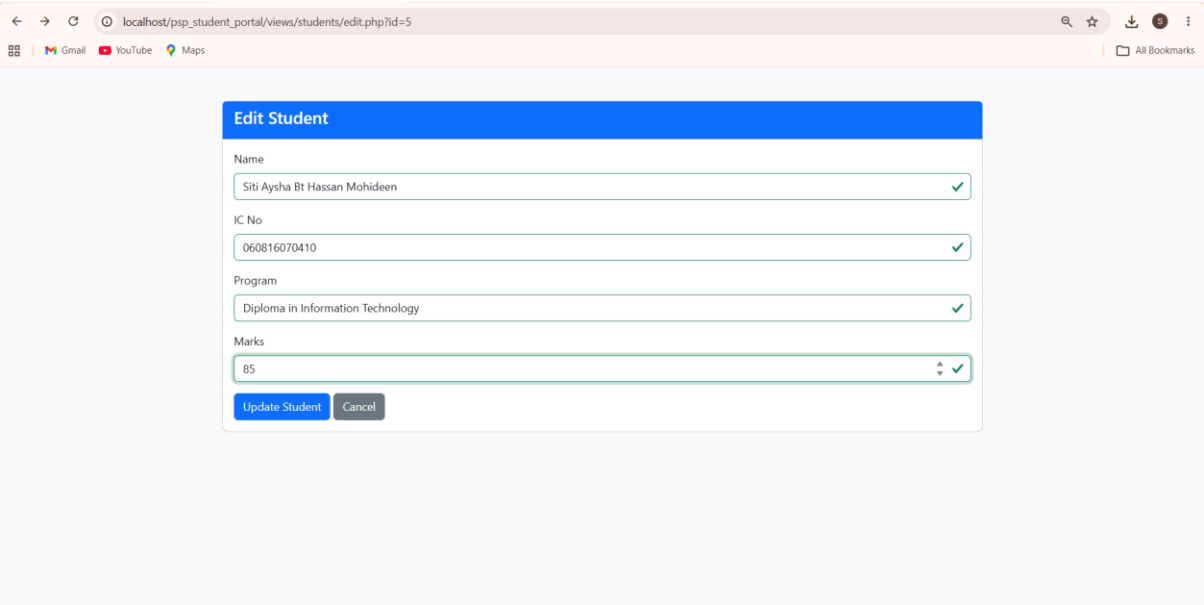
```
})();

</script>

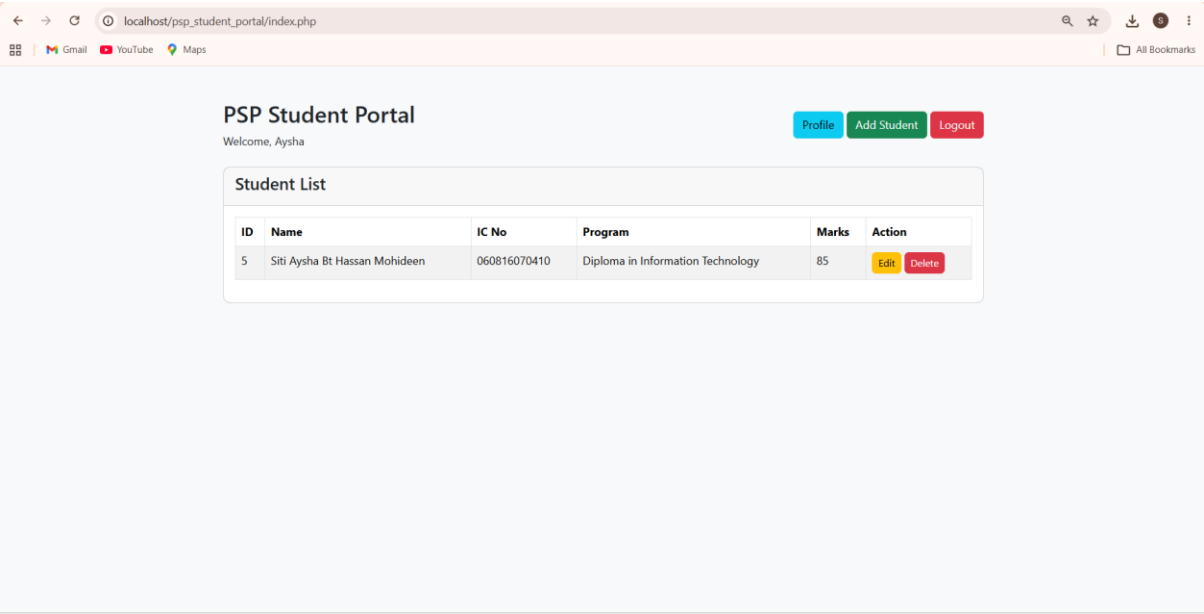
</body>

</html>
```

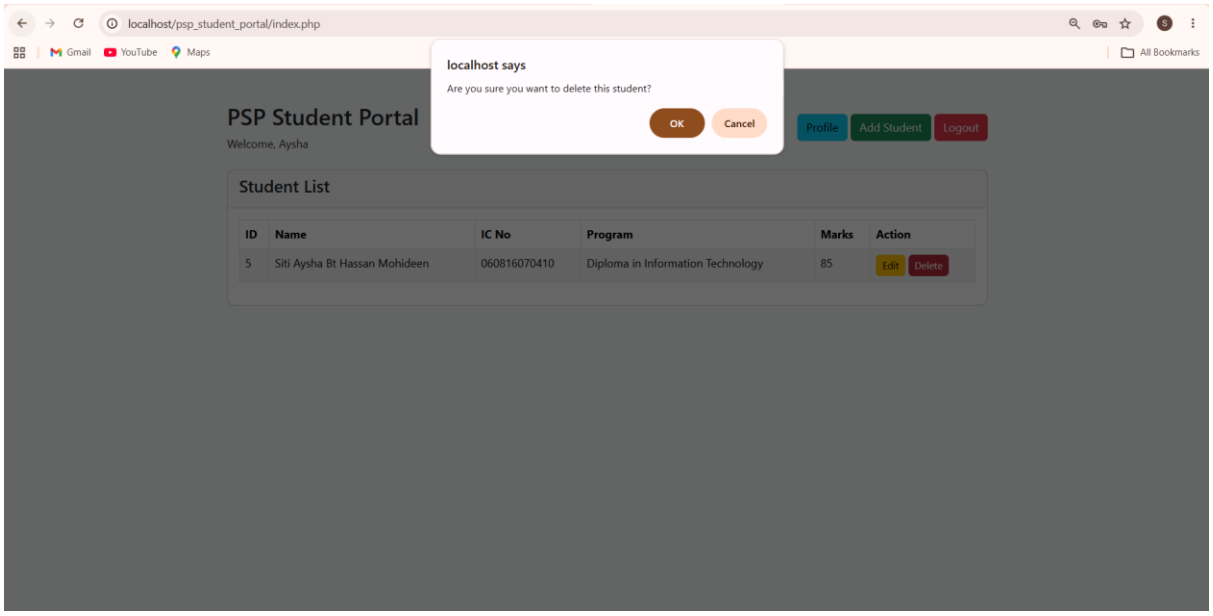
INTERFACE (AFTER edit marks 100 to 85)(Edit Student)



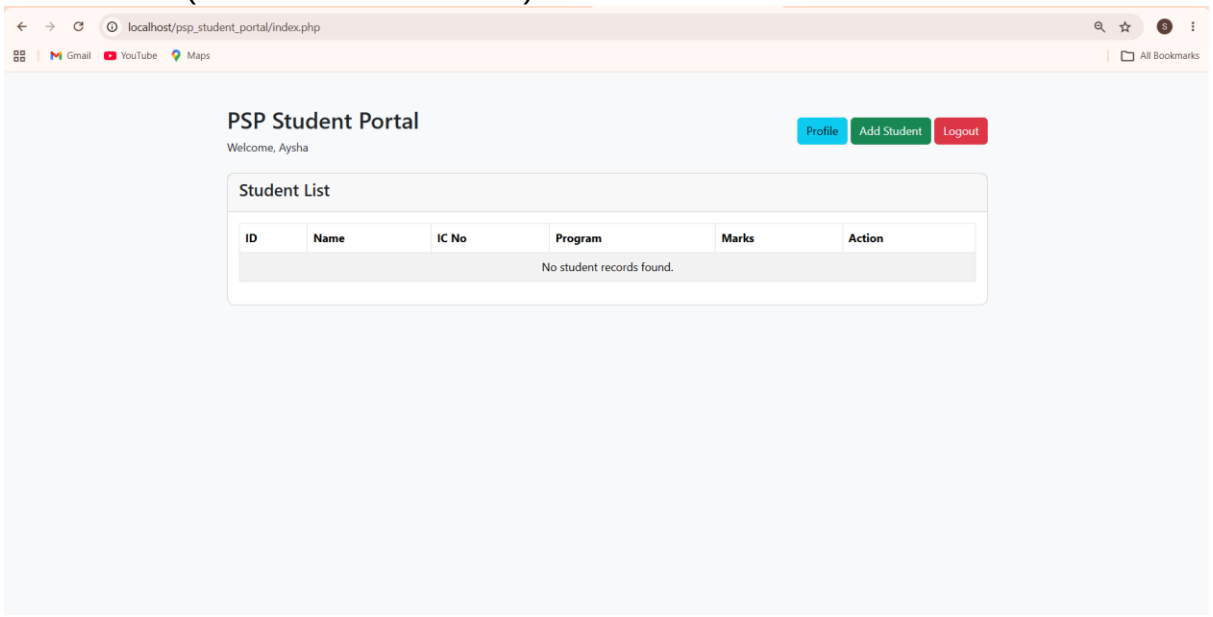
(Updated Student)



INTERFACE(Delete Confirmation)



INTERFACE(after delete Student list)



CODING views/profile.php

views > profile.php

```php
<?php

if (!isset($data)) {

    die("Profile data not found.");

}

?>

<!DOCTYPE html>

<html>

<head>

    <title>My Profile</title>

    <meta charset="UTF-8">

    <meta name="viewport"
        content="width=device-width, initial-scale=1">

    <link
        href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
        rel="stylesheet">

    <link
        rel="stylesheet"
        href="../public/css/style.css">

</head>

<body>

```

views > profile.php

```php

 <div class="container mt-5">

     <div class="row justify-content-center">

         <div class="col-md-7">

             <div class="card">

                 <div class="card-header">

                     <h4>My Profile</h4>

                 </div>

                 <div class="card-body">

                     <p>

                         <strong>Name:</strong>

                         <?php
                         echo htmlspecialchars($data['name']);
                         ?>

                     </p>

                     <p>

                         <strong>NRIC:</strong>

                         <?php
                         echo htmlspecialchars($data['icno']);
                         ?>

```

```
                    </p>

                    <p>

                        <strong>Program:</strong>

                        <?php
                        echo htmlspecialchars($data['program']);
                        ?>

                    </p>

                    <a
                        href="../views/change_password.php"
                        class="btn btn-warning">

                        Change Password

                    </a>

                    <a
                        href="../index.php"
                        class="btn btn-secondary">

                        Back

                    </a>

```

```
                </div>

            </div>

        </div>

    </div>

</div>

</body>

</html>

```

CODING AND INTERFACE controllers/ProfileController.php

CODING

controllers > ProfileController.php

```php
<?php

session_start();

include '../config/database.php';
include '../models/User.php';

if (!isset($_SESSION['user_id'])) {

    header("Location: ../views/login.php");
    exit;

}

$user = new User($conn);

$userId = $_SESSION['user_id'];

$data = $user->getStudentProfile($userId);

if (!$data) {

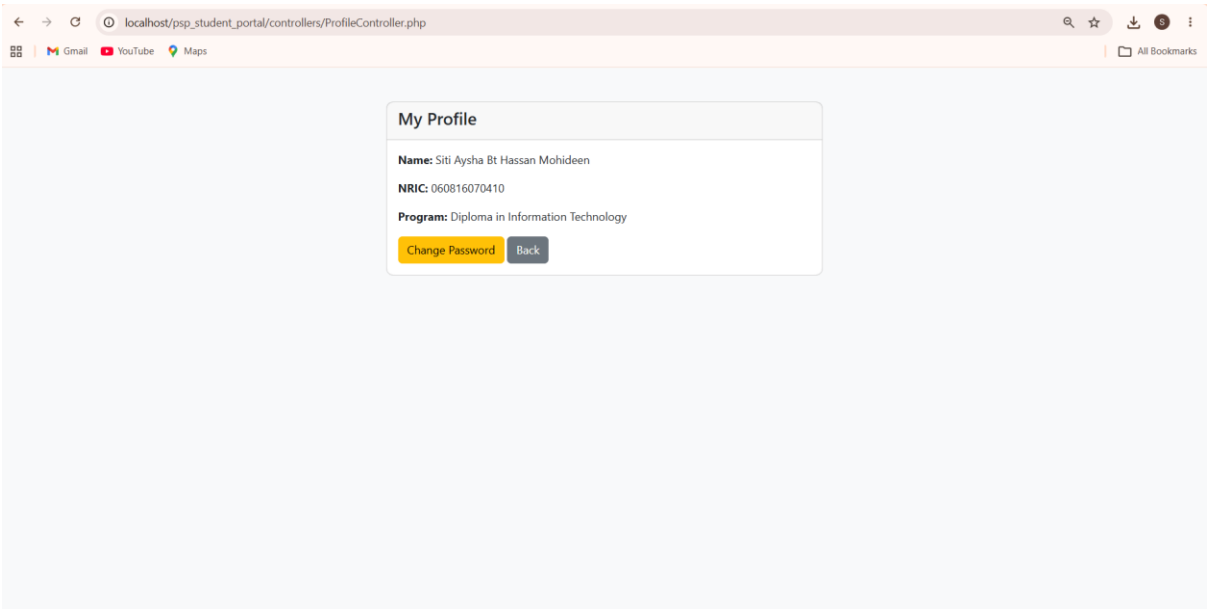
    die("Student profile not found.");

}

include '../views/profile.php';

?>
```

INTERFACE (Profile)



CODING controller/PasswordController.php

controllers > PasswordController.php

```php
<?php

session_start();

include '../config/database.php';
include '../models/User.php';

if (!isset($_SESSION['user_id'])) {

    header("Location: ../views/login.php");
    exit;

}

$user = new User($conn);

$userId = $_SESSION['user_id'];

if ($_SERVER['REQUEST_METHOD'] != 'POST') {

    header("Location: ProfileController.php");
    exit;

}

$currentPassword = $_POST['current_password'] ?? '';
$newPassword = $_POST['new_password'] ?? '';
$confirmPassword = $_POST['confirm_password'] ?? '';

if (
    $currentPassword == '' ||
    $newPassword == '' ||
    $confirmPassword == ''
) {

```


controllers > PasswordController.php
) {
die("Please fill in all fields.");
}
if ($newPassword != $confirmPassword) {
die("New password and confirm password do not match.");
}
if (strlen($newPassword) < 6) {
die("New password must be at least 6 characters.");
}
$data = $user->getById($userId);
if (!$data) {
die("User not found.");
}
if (!password_verify(
$currentPassword,
$data['password']
)) {
die("Current password is incorrect.");
}


}
if ($user->updatePassword(
$userId,
$newPassword
)) {
header("Location: ProfileController.php");
exit;
} else {
die("Failed to update password.");
}
?>

INTERFACE AND CODING views/change_password.php

CODING

views > change_password.php

```php
<?php

session_start();

if (!isset($_SESSION['user_id'])) {

    header("Location: login.php");
    exit;

}

?>

<!DOCTYPE html>

<html>

<head>

    <title>Change Password</title>

    <meta charset="UTF-8">

    <meta name="viewport"
        content="width=device-width, initial-scale=1">

    <link
        href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
        rel="stylesheet">

    <link
        rel="stylesheet"
        href="../public/css/style.css">

</head>
```

```
<body>

<div class="container mt-5">

    <div class="row justify-content-center">

        <div class="col-md-6">

            <div class="card">

                <div class="card-header">

                    <h4>Change Password</h4>

                </div>

                <div class="card-body">

                    <form
                        action="../controllers/PasswordController.php"
                        method="POST">

                        <div class="mb-3">

                            <label class="form-label">
                                Old Password
                            </label>

                            <input
                                type="password"
                                name="current_password"
                                class="form-control"
                                required>
```

```
                        </div>

                        <div class="mb-3">

                            <label class="form-label">
                                New Password
                            </label>

                            <input
                                type="password"
                                name="new_password"
                                class="form-control"
                                minlength="6"
                                required>

                        </div>

                        <div class="mb-3">

                            <label class="form-label">
                                Confirm Password
                            </label>

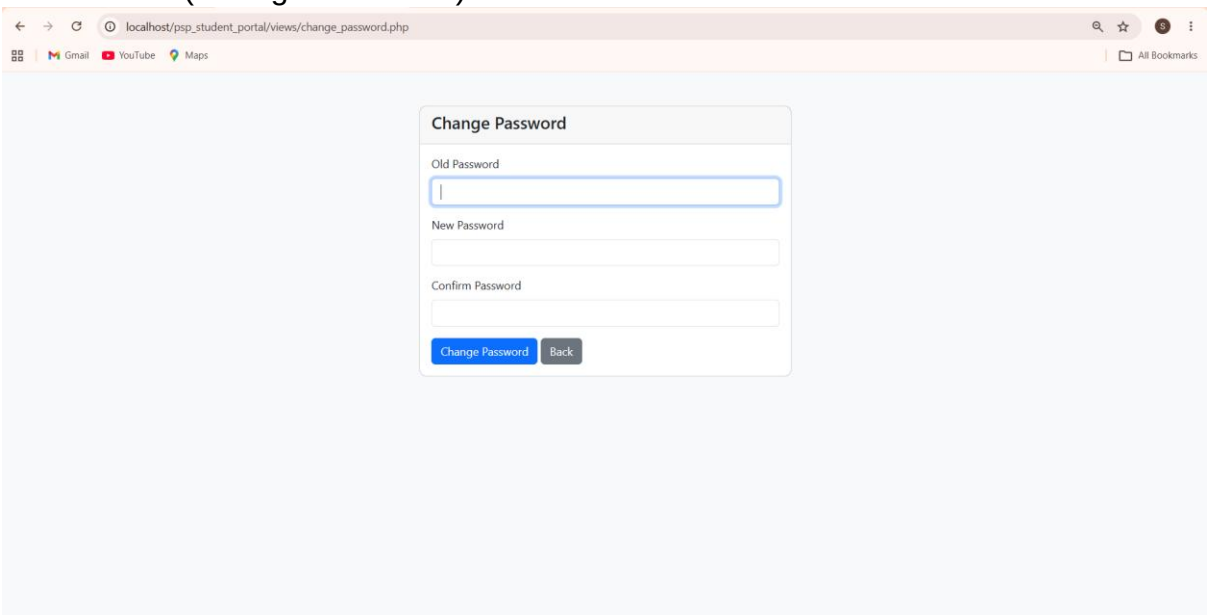
                            <input
                                type="password"
                                name="confirm_password"
                                class="form-control"
                                minlength="6"
                                required>
```

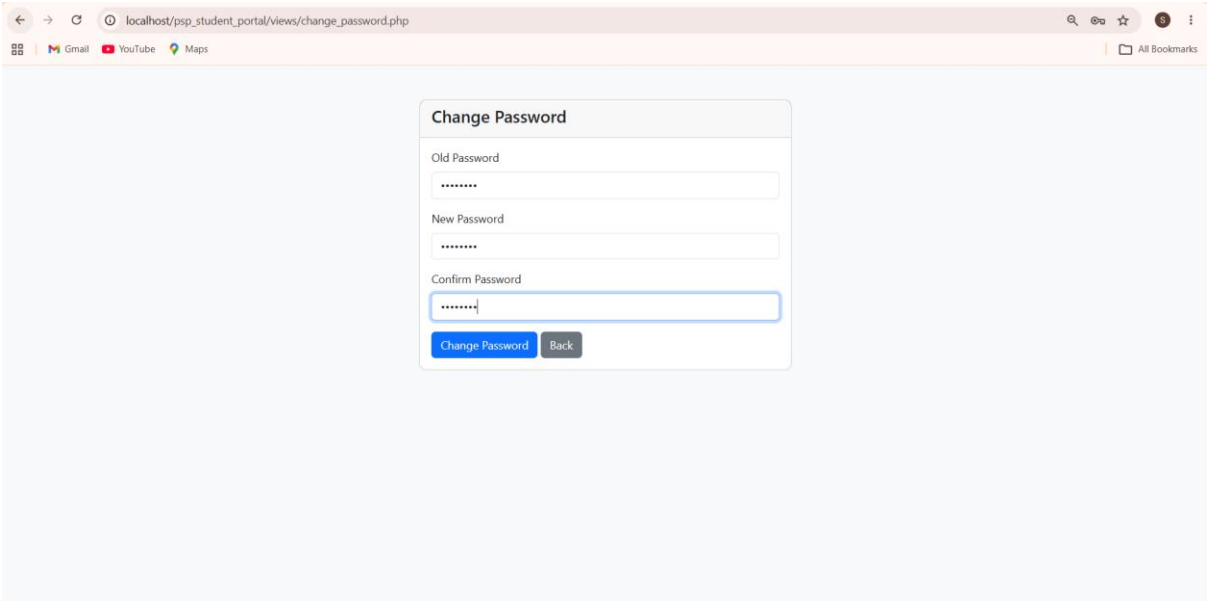
```
100
101                     </div>
102
103                     <button
104                         type="submit"
105                         class="btn btn-primary">
106
107                         Change Password
108
109                     </button>
110
111                     <a
112                         href="../controllers/ProfileController.php"
113                         class="btn btn-secondary">
114
115                         Back
116
117                     </a>
118
119                 </form>
120
121             </div>
122
123         </div>
124
125     </div>
126
127 </div>
128
129 </div>
```

views > change_password.php

```
129 </div>
130
131 </body>
132
133 </html>
```

INTERFACE(Change Password)





CODING models/Student.php

```php
<?php

class Student
{
    private $conn;

    public function __construct($conn)
    {
        $this->conn = $conn;
    }

    public function getAll()
    {
        $sql = "SELECT *
                FROM students
                ORDER BY id DESC";

        $stmt = $this->conn->prepare($sql);
        $stmt->execute();

        return $stmt->get_result();
    }

    public function getById($id)
    {
        $sql = "SELECT *
                FROM students
                WHERE id = ?";

        $stmt = $this->conn->prepare($sql);
        $stmt->bind_param("i", $id);
        $stmt->execute();

        return $stmt->get_result()->fetch_assoc();
    }
```

```php
    }

    public function getByIdAndUser($id, $userId)
    {
        $sql = "SELECT *
                FROM students
                WHERE id = ?
                AND user_id = ?";

        $stmt = $this->conn->prepare($sql);
        $stmt->bind_param("ii", $id, $userId);
        $stmt->execute();

        return $stmt->get_result()->fetch_assoc();
    }

    public function create($userId, $name, $icno, $program, $marks)
    {
        $sql = "INSERT INTO students
                (user_id, name, icno, program, marks)
                VALUES (?, ?, ?, ?, ?)";

        $stmt = $this->conn->prepare($sql);

        $stmt->bind_param(
            "isssi",
            $userId,
            $name,
            $icno,
            $program,
            $marks
        );

        return $stmt->execute();
```

```php
        return $stmt->execute();
    }

    public function update(
        $id,
        $userId,
        $name,
        $icno,
        $program,
        $marks
    ) {
        $sql = "UPDATE students
                SET name = ?,
                    icno = ?,
                    program = ?,
                    marks = ?
                WHERE id = ?
                AND user_id = ?";

        $stmt = $this->conn->prepare($sql);

        $stmt->bind_param(
            "sssiii",
            $name,
            $icno,
            $program,
            $marks,
            $id,
            $userId
        );

        return $stmt->execute();
    }
```

```php
    }

    public function delete($id, $userId)
    {
        $sql = "DELETE FROM students
                WHERE id = ?
                AND user_id = ?";

        $stmt = $this->conn->prepare($sql);

        $stmt->bind_param(
            "ii",
            $id,
            $userId
        );

        return $stmt->execute();
    }
}

?>

```

CODING models/User.php

models > User.php

```php
<?php

class User
{
    private $conn;

    public function __construct($conn)
    {
        $this->conn = $conn;
    }

    public function getById($id)
    {
        $sql = "SELECT *
                FROM users
                WHERE id = ?";

        $stmt = $this->conn->prepare($sql);
        $stmt->bind_param("i", $id);
        $stmt->execute();

        return $stmt->get_result()->fetch_assoc();
    }

    public function getByEmail($email)
    {
        $sql = "SELECT *
                FROM users
                WHERE email = ?";

        $stmt = $this->conn->prepare($sql);
        $stmt->bind_param("s", $email);
        $stmt->execute();

        return $stmt->get_result()->fetch_assoc();
```

```php
        return $stmt->get_result()->fetch_assoc();
    }

    public function getStudentProfile($userId)
    {
        $sql = "SELECT
                    users.id,
                    users.username,
                    users.email,
                    students.name,
                    students.icno,
                    students.program,
                    students.marks
                FROM users
                INNER JOIN students
                    ON users.id = students.user_id
                WHERE users.id = ?";

        $stmt = $this->conn->prepare($sql);
        $stmt->bind_param("i", $userId);
        $stmt->execute();

        return $stmt->get_result()->fetch_assoc();
    }

    public function updatePassword($id, $password)
    {
        $hashedPassword = password_hash($password, PASSWORD_DEFAULT);

        $sql = "UPDATE users
                SET password = ?
                WHERE id = ?";
```

```php
        $stmt = $this->conn->prepare($sql);
        $stmt->bind_param("si", $hashedPassword, $id);

        return $stmt->execute();
    }
}

?>
```

CODING public/css/style.css

public > css > # style.css > .card

```css
body {
    background-color: #f8f9fa;
}

.container {
    max-width: 1100px;
}

.card {
    border-radius: 10px;
}

.card-header {
    font-weight: bold;
}

.btn {
    border-radius: 6px;
}
```

CODING public/css/style.css

public > js > JS script.js

```
document.addEventListener("DOMContentLoaded", function () {

    console.log("PSP Student Portal loaded successfully.");

});
```

reset_password.php

reset_password.php

```
<?php

include 'config/database.php';

$password = "Aysha@1686";

$hash = password_hash($password, PASSWORD_DEFAULT);

$sql = "UPDATE users SET password = ? WHERE id = 1";

$stmt = $conn->prepare($sql);
$stmt->bind_param("s", $hash);

if ($stmt->execute()) {

    echo "Password updated successfully.";

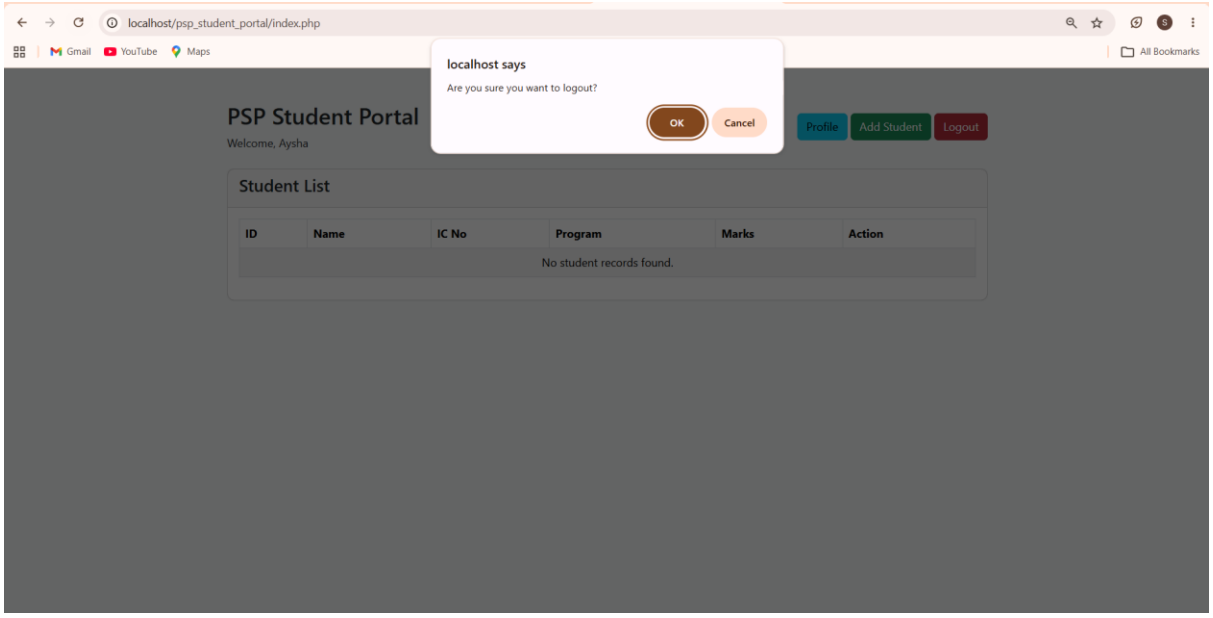
} else {

    echo "Failed to update password.";

}

?>
```

INTERFACE(Logout)



DATABASE SCREENSHOT

psp_student_portal

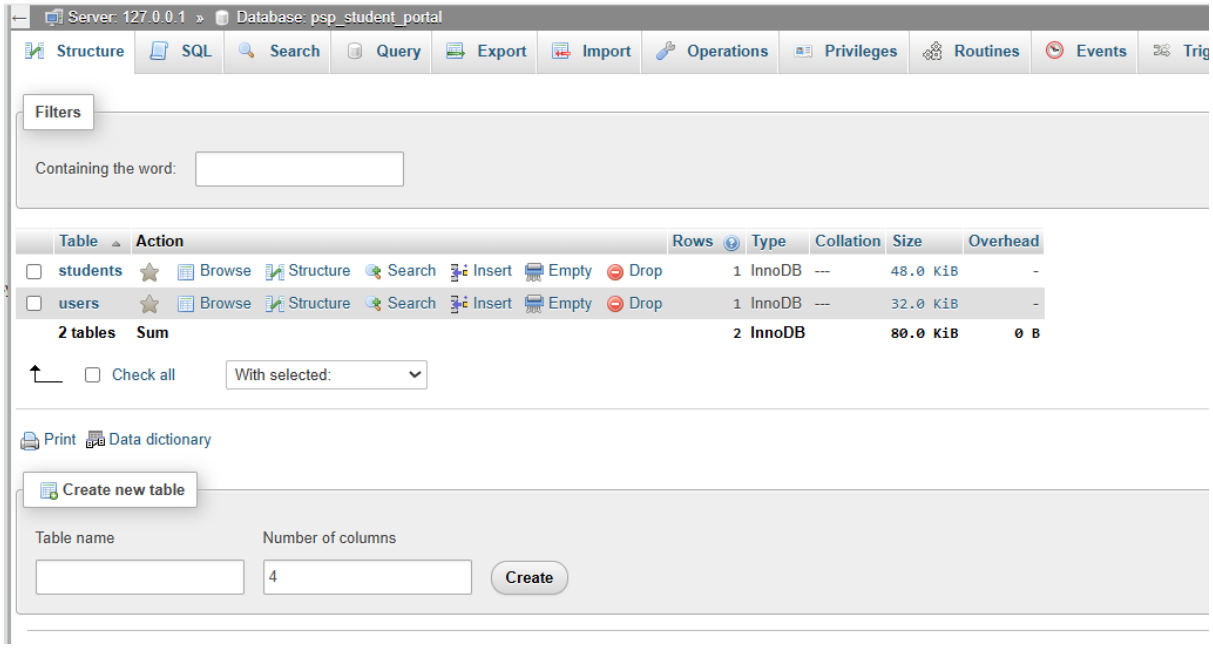


Table:student

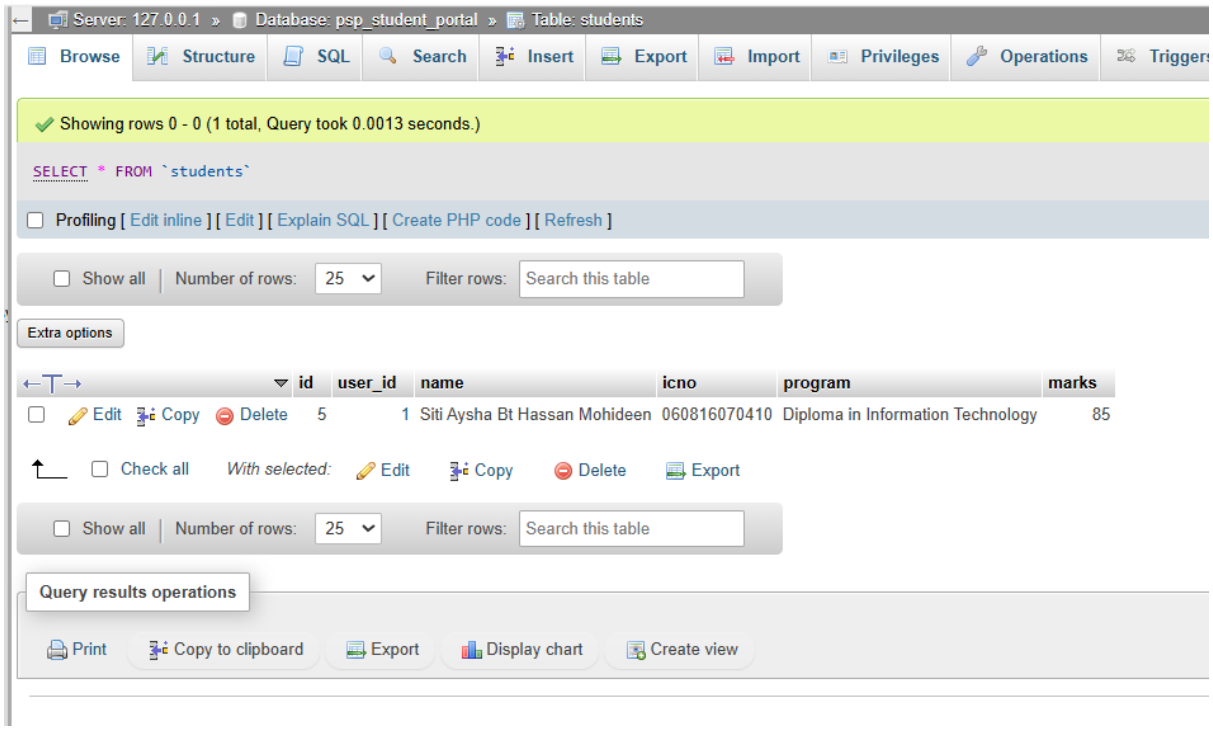
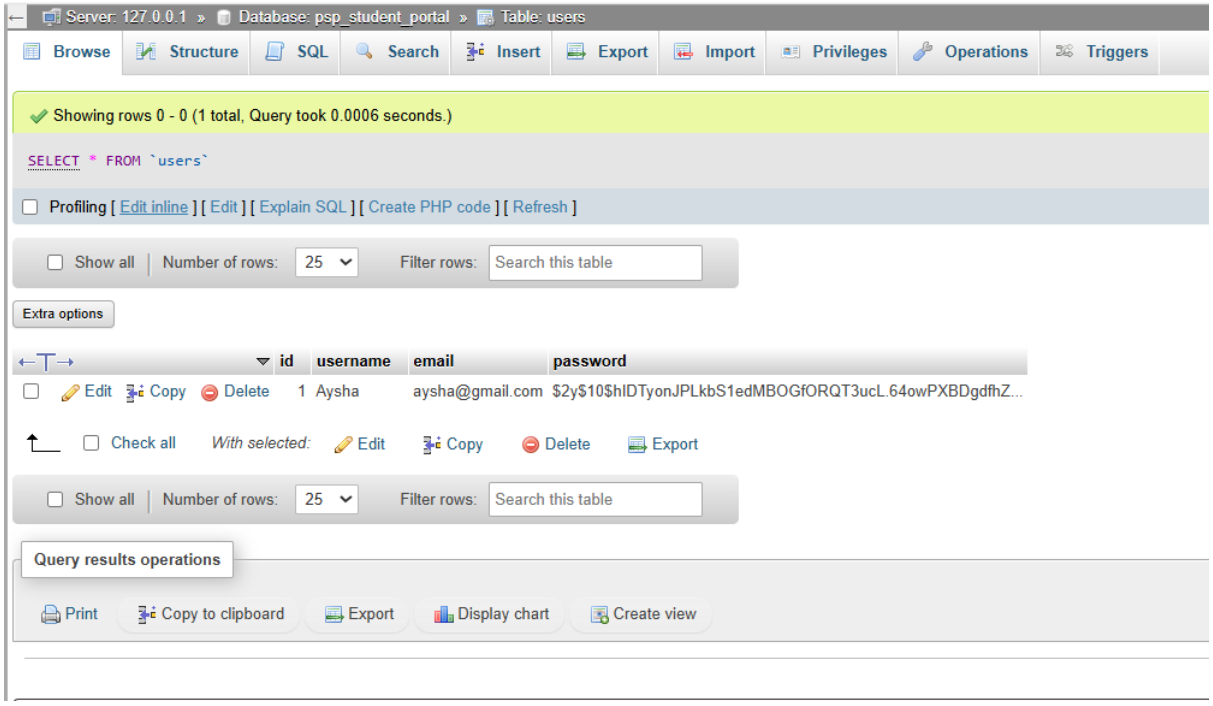


Table:User



21. CONCLUSION

The Web Based PSP Student Portal successfully demonstrates the implementation of a PHP and MySQL web application using MVC architecture.

The system provides important functions including login authentication, profile management, password management and student CRUD operations.

The project also demonstrates secure coding practices. Passwords are protected using password_hash() and password_verify(). Database operations use prepared statements, while session management protects authenticated pages.

The system implements both client-side and server-side validation. Access control is also applied to ensure that users can only modify authorised student records.

The use of MVC architecture separates the Model, View and Controller responsibilities. This makes the application more organised, easier to maintain and easier to extend.

Overall, the project demonstrates the practical application of full stack web development concepts using PHP, MySQL, Bootstrap, HTML, CSS and JavaScript. The completed system fulfils the major functional and technical requirements of the Web Based PSP Student Portal practical assessment.

INFORMATION AND COMMUNICATION TECHNOLOGY DEPARTMENT
MINI PROJECT 1 RUBRIC | **SESI 1 2026/2027**

COURSE CODE	**DFP40443**		**COURSE NAME**	**Full Stack Web Development**			
ASSESSMENT TASK	**Mini Project 1**		**LECTURER'S NAME**				
STUDENT'S NAME	**1**		**REGISTRATION NUMBER**	**1**			
	2			**2**			
	3			**3**			
	4			4			
Category / Score	**4 (Excellent)**	**3 (Good)**	**2 (Fair)**	**1 (Poor)**	**Weight (%)**	**Student's Score (out of 4)**	**Marks (Score × Weight / 4)**
MVC & Modular Structure	Clear MVC separation; highly modular code using include/require.	MVC used but with minor organization issues.	Limited modularity; logic and view are not fully separated.	No MVC structure; unorganized code.	**10**		0
Session & Profile (Q1)	Profile functions perfectly via $_SESSION; strict access control.	Session works, but minor flaws in access control logic.	Profile displays data but session management is unstable.	Failed to display information based on active sessions.	**15**		0
Password Security (Q1)	Accurate old password validation and perfect hashing implementation.	Hashing is used but validation messages are unclear.	Update works but lacks hashing or old password validation.	Passwords stored in plain text; no security measures.	**15**		0
Database & CRUD (Q2)	All CRUD operations work perfectly; seamless migration from array to SQL.	CRUD works but with minor bugs in one function (e.g., Update).	CRUD is partially complete with several logic errors.	Failed to connect to MySQL; CRUD is non-functional.	**25**		0
SQL Security	Consistently uses Prepared Statements for	Uses Prepared Statements for most	Queries work but are vulnerable to SQL Injection (raw variables).	No security considerations in SQL code.	**10**		0

	all database interactions.	queries with minor omissions.					
Technical Documentation	Complete report: ERD, MVC structure, functional screenshots, and deep technical explanations.	Complete report but lacking detailed technical descriptions	Report provided but disorganized or missing key screenshots.	Very brief documentation; missing format or evidence of testing.	**25**		0
						Total score	**0**